

Week 8

Ansible – Configuration Management



Ansible Concepts

-  Why Ansible Matters?
-  Ansible vs Python vs Shell
-  How Ansible Works?
-  Installing Ansible on Linux
-  Installing Ansible on Window
-  Setting Up Your Dev Environment
-  Setting Up SSH Access
-  Understanding Inventory

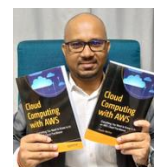
Ansible Concepts

-  Configure Passwordless SSH using Terraform
-  Ad-Hoc Commands
-  Writing Your First Playbook
-  Multiple Plays
-  Deploy Mini Finance Project using Terraform & Ansible
-  Deploy React Application using Terraform & Ansible
-  When to Use Ansible Roles
-  Understanding Role Structure

Ansible Concepts

-  Deploy Mini Finance Project using Terraform and Ansible Roles
-  Full Automation The EpicBook App using Terraform + Ansible on Azure Cloud

Why Ansible Matters?

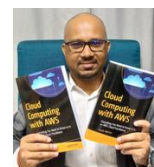


AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

Why Ansible Matters?

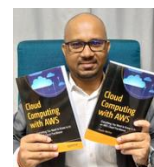
Problem with **Manual Server Management**

- Slow
- Error prone
- Inconsistent
- Impossible to scale reliably



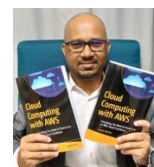
From Manual Tasks to Automation

-  Installing and updating software **packages**
-  Creating users and **managing permissions**
-  **Configuring services** like web servers and databases
-  Applying **security patches** and **firewall rules**



The Problem with Shell Scripts

-  Shell scripts are **platform-specific**.
-  They're **fragile** and **hard to maintain**
-  They lack **idempotency**
-  There's no structure or built-in error handling.



The Rise of Configuration Management Tools

-  **Define** infrastructure and system configurations in a **repeatable, scalable way**
-  Most required **agents to be installed** on every machine
-  The learning curve was **steep**.

Tool	Characteristics
Puppet	Declarative, uses its own DSL, agent-based
Chef	Ruby-based DSL, agent-based, customizable
SaltStack	YAML-based configs, master-agent model

Why Ansible Changed Everything

 Make automation **simple**

Feature	Why It Matters
Agentless	No agents required. Uses SSH or WinRM—already available.
YAML Playbooks	Human-readable. No coding background needed.
Idempotent	Safe to run tasks repeatedly. No duplication or side effects.
Cross-platform	Works on Linux, macOS, and Windows.
Modular & Scalable	Use ad-hoc tasks, roles, plugins, or complex playbooks.

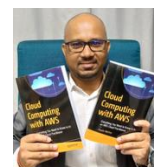
Ansible vs Python vs Shell



AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

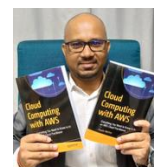
Shell Scripts: Quick but **Risky**

-  Installing a package
-  Moving a file
-  Restarting a service
-  Are **not cross-platform**
-  Aren't **idempotent**
-  Get **messy quickly** when tasks grow in complexity
-  Offer little in terms of **error handling** or **logging**



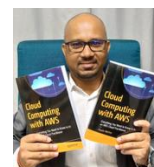
Python Scripts: Flexible but **Requires More Effort**

-  Make API calls
-  Use loops and conditionals
-  Parse and transform data
-  Build reusable modules



So When Should You Use Ansible?

-  You want to define what the **end state should be** (e.g., “Java 11 must be installed”)
-  You want **idempotent**, repeatable automation
-  You’re managing **multiple machines** or **environments**
-  You want something **declarative** and **readable** that your whole team can understand



How Ansible Works?



What Makes Ansible Different?

-  **Agentless** by Design
-  **Written in YAML**, Not Code
-  **SSH** (for Linux/macOS) and **WinRM** (for Windows)

```
- name: Install nginx
  apt:
    name: nginx
    state: present
```

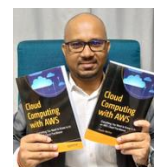
How Ansible Works in Practice?

The Control Node (**The Brain of Your Automation**)

- Your playbooks
- Your inventory files (**list of servers**)
- The Ansible binary **itself**

The Managed Nodes (**The Machines You Control**)

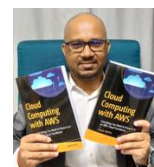
- Linux virtual machines
- Windows servers
- Cloud instances (like EC2, Azure VMs, etc.)



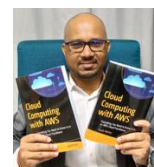
How Ansible Communicates?

 SSH for Linux and macOS

 WinRM for Windows

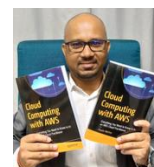


Install Ansible on Linux



AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

Install Ansible on Window



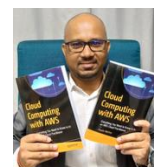
AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

Setting Up Your Dev Environment

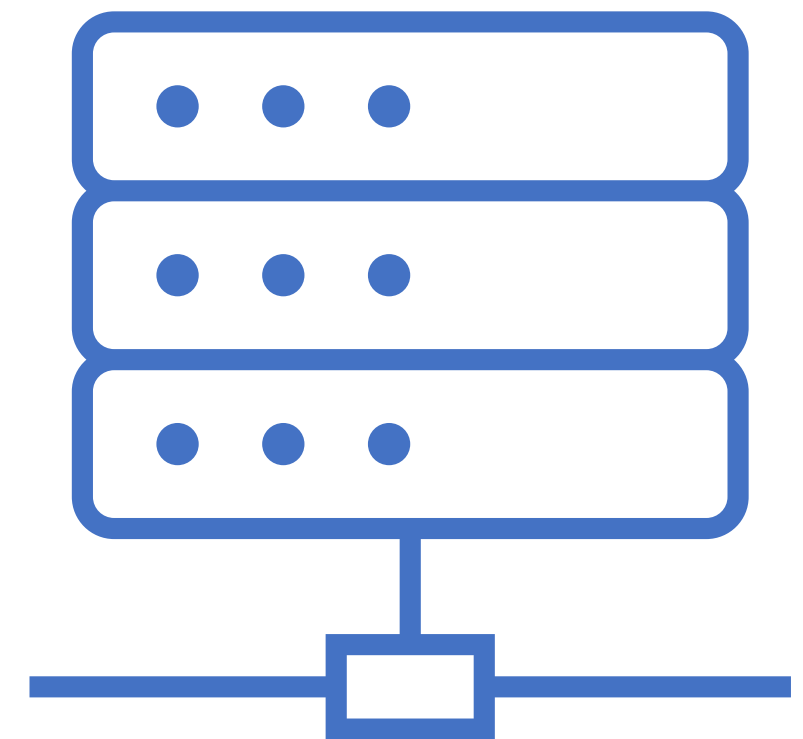
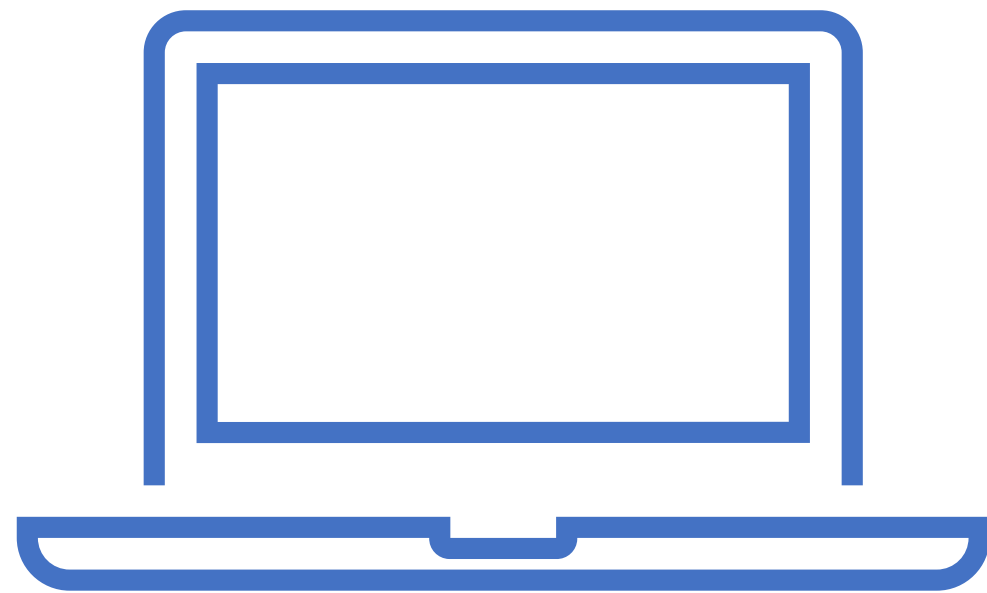


AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

Setting Up SSH Access



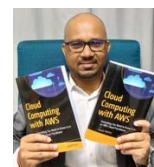
AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>



Two Ways to Set Up SSH Access

-  SSH Key-Based Authentication (**Recommended**)
-  Password-Based Authentication (**Not ideal for automation**)
-  A **private key** stays with you (**never shared**)
-  A **public key** is placed on the managed node
-  `ssh-keygen`

Understanding Inventory

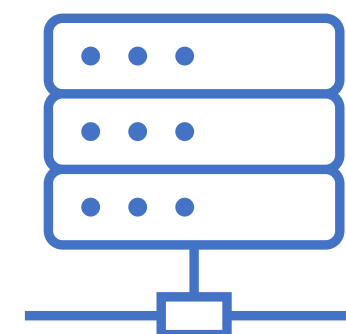
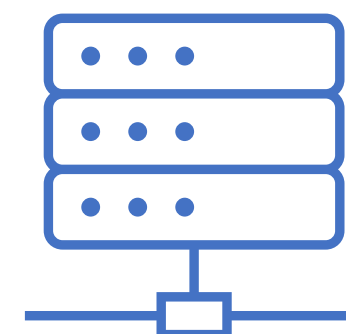
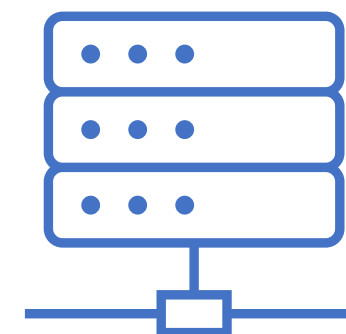
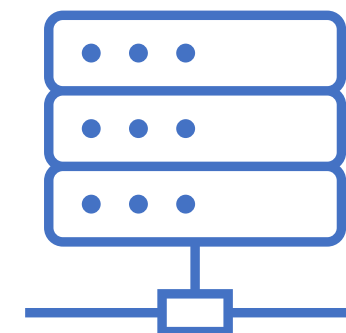
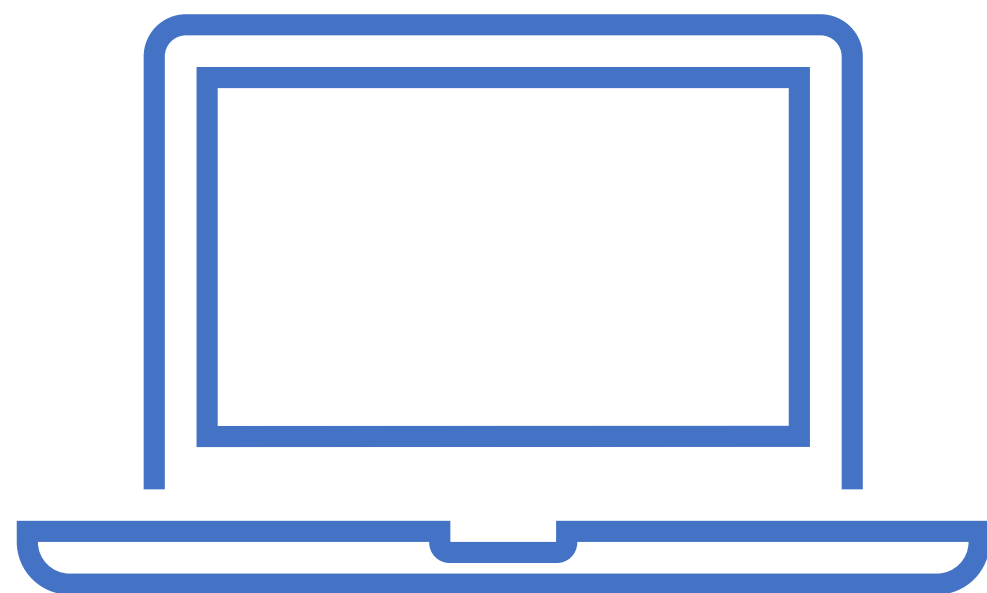


AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

What Is Ansible Inventory?

 The Ansible Inventory is **simply a file** (or **a script**) that lists **all the hosts** you want to manage.

- What are the IP addresses?
- What groups?
- What connection details or variables should Ansible use?



Static vs Dynamic Inventory

Static Inventory

- **Controlled** environments
- Written in **.ini** or **.yaml** format
- Define what **groups** exist

Dynamic Inventory

- Pull the list of hosts **automatically**
- List all VMs tagged with **env=production**

Example: **.ini** format

 **.ini** format (**default** and **widely** used)

```
[web]
```

```
192.168.1.10
```

```
192.168.1.11
```

```
[db]
```

```
192.168.1.20
```

```
[all:vars]
```

```
ansible_user=azureuser
```

```
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

Example: **.yaml format**

 **.yaml** format (more structured, **especially for larger environments**)

```
all:
  hosts:
    web1:
      ansible_host: 192.168.1.10
    web2:
      ansible_host: 192.168.1.11
    db1:
      ansible_host: 192.168.1.20
  vars:
    ansible_user: azureuser
    ansible_ssh_private_key_file: ~/.ssh/id_rsa
```

Best Practices

 /etc/ansible/hosts

 Use a custom inventory file inside your project (**inventory.ini**)

 Organizing Hosts into **Groups**

```
[app]  
10.0.0.10  
10.0.0.11
```

```
[db]  
10.0.0.20
```

```
[cache]  
10.0.0.30
```

```
[production:children]  
app  
db
```

Configure Passwordless SSH using Terraform



AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

Ad-Hoc Commands



AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

What Are Ad-Hoc Commands?

-  Ad-Hoc commands are **one-line instructions**
-  Quick actions, One-time jobs, Infrastructure debugging tools
-  Pinging a group of servers, Rebooting a VM, Installing a package, Copying a config file, Checking system uptime
-  `ansible all -i inventory.ini -m <module_name> -a "<arguments>"`
-  `ansible web -i inventory.ini -m ping`
-  `ansible all -i inventory.ini -m shell -a "uptime"`
-  `ansible app -i inventory.ini -m apt -a "name=nginx state=present" --become`
-  `ansible db -i inventory.ini -m service -a "name=mysql state=restarted" --become`



Example: **inventory.ini**

```
[web]
```

```
192.168.1.10
```

```
192.168.1.11
```

```
[db]
```

```
192.168.1.20
```

```
[all:vars]
```

```
ansible_user=azureuser
```

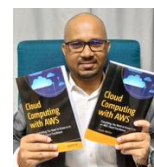
```
ansible_ssh_private_key_file=~/.ssh/id_rsa
```



When to Use Ad-Hoc Commands vs Playbooks

Use Case	Best Tool
One-time task	Ad-Hoc Command
Something you'll reuse	Playbook
Complex logic with variables and handlers	Playbook
Easily readable documentation of a workflow	Playbook
Automation inside CI/CD	Playbook

Writing Your First Playbook



AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

Why Playbooks need?

-  Ad-Hoc commands are **fast**, **spontaneous**, and great for **one-off tasks**.
-  You want a **blueprint** & a **repeatable checklist**.
-  A playbook is a **YAML file** that describes:
 -  What you want to automate?
 -  In what order?
 -  On which servers?
 -  And using which privileges?

What's Inside a Playbook?

 A playbook is made up of one or more plays, and each play tells Ansible:

- Which **hosts** to target?
- What **tasks** to run
- Whether to **use sudo** (become)
- Which **remote user** to connect as
- Which **modules** to use in each task

```
- name: Install and start nginx
  hosts: web
  become: yes
  remote_user: azureuser

  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present

    - name: Start nginx
      service:
        name: nginx
        state: started
```



How Does Ansible Know What to Do?



Each task in your playbook uses a module — like **apt**, **service**, **copy**, or **template** — to perform an action.

```
- name: Install nginx
  apt:
    name: nginx
    state: present
```

Multiple Plays



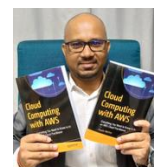
AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

Deploy Mini Finance Project Using Terraform & Ansible



AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

When to Use Ansible Roles?



The Automation Is Growing

 Deploy **real application** Mini Finance to **real servers**.

 You have:

- **Installed** Nginx
- **Started** and **Enabled** Nginx
- **Clone** the Mini Finance project from GitHub
- **Copy** website content to web server root
- **Overwrite** default Nginx site configuration
- **Restart** Nginx After Configuration

What's Inside a Playbook?

-  Install **Nginx**, **Node.js** *and* **Git**?
-  Set up **firewalls**?
-  Configure **environment variables**?
-  Deploy applications with **templated config files**?
-  Do this on ***multiple types*** of servers across ***multiple environments***?
-  It's not long before your playbook grows from **20 lines to 200... or even 1,000+.**

What is the Problem with Large Playbooks?

-  Hard to **read**
-  Hard to **maintain**
-  Impossible to **reuse**
-  Fragile to **collaboration**

What Is a Role?

 A role is a **self-contained** unit of configuration.

- Modular
- Reusable
- Organized

Real-World Example: **Mini Finance**

-  **common:** apply system updates, install basic packages
-  **nginx:** install and configure Nginx
-  **mini_finance:** clone the Mini Finance project and deploy the website
-  **firewall:** (optional) configure UFW or iptables to allow port 80

Understanding Ansible Role Structure



AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>



Create Your First Role



ansible-galaxy init nginx

```
nginx/
├── defaults/
├── files/
├── handlers/
├── meta/
├── tasks/
├── templates/
├── vars/
└── README.md
```

```
- hosts: web
  become: yes
  roles:
    - common
    - nginx
    - mini_finance
```

Folder	Think of it as...	What it contains
tasks/	The core engine	The main tasks your role performs
handlers/	Emergency responders	Actions triggered when something changes
defaults/	Friendly suggestions	Default variable values, easily overridden
vars/	Internal constants	Higher-priority variables
files/	Static assets	Files copied as-is to remote servers
templates/	Dynamic documents	Config files with placeholders (Jinja2)
meta/	The role's passport	Role dependencies, metadata, author info

tasks/ – The Heart of the Role

 You'll always have a **main.yml** inside the **tasks/** folder

```
# roles/nginx/tasks/main.yml
- name: Install nginx
  apt:
    name: nginx
    state: present

- name: Ensure nginx is running
  service:
    name: nginx
    state: started
    enabled: yes
```

handlers/ – React When Something Changes

 only run **when notified**

 Handlers give you **controlled execution**

You define them like this:

```
# roles/nginx/handlers/main.yml
- name: restart nginx
  service:
    name: nginx
    state: restarted
```

Then you call them from tasks like this:

```
- name: Copy nginx config
  copy:
    src: nginx.conf
    dest: /etc/nginx/nginx.conf
  notify: restart nginx
```

defaults/ vs vars/ – Know the Difference

 **Both** of these folders contain variables — but with a important difference in **priority**

 **defaults/ – Lowest Priority**

 **vars/ – Higher Priority**

```
# roles/nginx/defaults/main.yml
nginx_port: 80
```

```
# roles/nginx/vars/main.yml
nginx_package_name: nginx
```

You want...	Use
Configurable values (e.g. port, document root)	defaults/
Core internal settings	vars/

files/ vs templates/ – Static vs Dynamic

 **files/ – Copy As-Is:** This is for files that don't need to be customized.

 **templates/ – Change on the Fly:** Inject variables into files — like config files or web pages.

```
# Copy a static script
- name: Copy startup script
  copy:
    src: startup.sh
    dest: /usr/local/bin/startup.sh
    mode: '0755'
```

Templates use **Jinja2** syntax:

```
# templates/nginx.conf.j2
server {
    listen {{ nginx_port }};
    server_name {{ nginx_hostname }};
    ...
}
```

And in your task:

```
- name: Generate nginx config
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
```

meta/ – Describe the Role Itself

-  Role author and description
-  Platform compatibility
-  Dependencies on other roles

```
# meta/main.yml
galaxy_info:
  author: yourname
  description: Installs and configures nginx
  platforms:
    - name: Ubuntu
      versions:
        - 20.04
        - 22.04
  dependencies: []
```

Do I Have to Use Every Folder?

 **Not at all.**

 In most real-world roles, you'll use:

- tasks/
- handlers/
- defaults/
- templates/ or files/

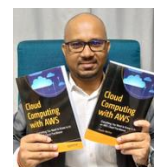
 The rest you can introduce as needed.

Deploy Mini Finance Project using Terraform and Ansible Roles



AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>

Deploy The EpicBook Project using Terraform and Ansible Roles



AWS with Pravin Mishra:
<https://university.pravinmishra.in/learn>